Desafio OPC UA: Monitoramento de Motor Trifásico

Objetivo

Desenvolver um servidor OPC UA completo para monitoramento industrial, integrando conceitos modernos de backend, como microsserviços e APIs RESTful.

Requisitos Principais:

- **Servidor OPC UA**: Suporte a Discovery Local, Alarmes e Eventos, Histórico de Variáveis, Histórico de Eventos e Nodeset personalizado.
- **Comunicação**: Integração com Broker MQTT para recebimento de dados.
- **Arquitetura**: Implementação em microsserviços (OPC UA, Ingestão MQTT, API RESTful).
- **Broker de Mensagens**: Uso do RabbitMQ para comunicação interna entre serviços.
- **API**: Exposição dos dados via API RESTful implementada com FastAPI, com documentação Swagger.
- **Testes**: Cobertura com testes unitários e de ponta a ponta (E2E).
- **Documentação**: Geração de documentação de código com Pydoc.
- **Visualização**: Sugestão de um cliente para visualização dos dados (ex: Grafana).

Cenário

Uma indústria precisa monitorar um motor trifásico de 50cv. Três sensores são responsáveis por coletar dados e enviá-los periodicamente via MQTT.

Sensores e Frequência de Envio

Categoria	Sensor	Variáveis	Frequência
Electrical	Sensor Elétrico	Tensão (3 fases), Corrente (3 fases), Potência (3), Energia (3), Fator de Potência, Frequência	A cada 5s
Environment	Sensor Ambiental	Temperatura, Umidade, Temperatura da Carcaça	A cada 5s

Categoria	Sensor	Variáveis	Frequência
Vibration	Sensor de Vibração	Vibração Axial, Vibração Radial	A cada 60s

Organização do Nodeset OPC UA

A estrutura de nós no servidor deve seguir o seguinte modelo:

```
Motor50CV
├── Electrical
│   ├── VoltageA
│   ├── VoltageB
│   ├── VoltageC
│   ├── CurrentA
│   ├── CurrentB
│   ├── CurrentC
│   ├── PowerActive
│   ├── PowerReactive
│   ├── PowerApparent
│   ├── EnergyActive
│   ├── EnergyReactive
│   ├── EnergyApparent
│   ├── PowerFactor
│   └── Frequency
├── Environment
│   ├── Temperature
│   ├── Humidity
│   └── CaseTemperature
└── Vibration
    ├── Axial
    └── Radial
```

Alarmes e Eventos

O servidor OPC UA deve gerar eventos e alarmes com as seguintes condições:

Evento	Categoria	Severidade	Mensagem
Tensão > 10% acima da nominal	Electrical	Alta	Overvoltage detected
Tensão < 10% abaixo da nominal	Electrical	Alta	Undervoltage detected
Corrente > 10% acima da nominal	Electrical	Alta	Overcurrent detected

Evento	Categoria	Severidade	Mensagem
Temperatura da carcaça > 60°C	Environment	Crítica	Case temperature critical

Tabela para os valores de criticidade

Criticidade	Valor de `Severity`	Uso no seu exercício
Informativo	`100`	Eventos de status, heartbeat, etc.
Baixa	`250`	Vibração leve ou pequenas variações
Média	`500`	Temperatura levemente acima do esperado
Alta	`700`	Sobrecorrente ou sobretensão moderada
Crítica	`900`	Temperatura da carcaça acima de 60°C

Histórico

- Armazene o histórico das variáveis elétricas e da temperatura da carcaça, respeitando a resolução de cada sensor.
- Armazene o histórico de eventos e alarmes em um banco de dados local (ex: SQLite) ou em arquivo.

Discovery

- Crie uma task no servidor OPC UA que faça o registro periódico em um LDS (Local Discovery Server).

Integração com MQTT

- **Broker Remoto**: `lse.dev.br`
- **Tópicos e Formato das Mensagens**:
  - **Tópico**: `scgdi/motor/electrical`

```
{
  "timestamp": "2025-08-08T12:00:00Z",
  "voltage": {"a": 220.0, "b": 221.1, "c": 219.5},
  "current": {"a": 10.5, "b": 10.4, "c": 10.6},
  "power": {"active": 4500, "reactive": 500, "apparent": 4600},
  "energy": {"active": 10000, "reactive": 1200, "apparent": 1020
  "powerFactor": 0.95,
```

```
  "frequency": 60.0
}
```

- **Tópico**: `scgdi/motor/environment`

```
{
  "timestamp": "2025-08-08T12:00:00Z",
  "temperature": 34.5,
  "humidity": 55.2,
  "caseTemperature": 62.0
}
```

- **Tópico**: `scgdi/motor/vibration`

```
{
  "timestamp": "2025-08-08T12:00:00Z",
  "axial": 0.12,
  "radial": 0.15
}
```

Tarefas Detalhadas

1. **Arquitetura**: Estruture o projeto em microsserviços (ex: `mqtt-ingestor`, `opcua-server`, `api-service`). Utilize Docker e Docker Compose para orquestrar os contêineres e o RabbitMQ.
2. **Serviço de Ingestão MQTT**: Implemente um cliente MQTT que se conecte ao broker, subscreva aos tópicos, valide os dados recebidos e os publique em uma fila do RabbitMQ.
3. **Serviço OPC UA**:
   - Crie o servidor OPC UA com a árvore de variáveis especificada.
   - Implemente um consumidor RabbitMQ para receber os dados do serviço de ingestão e atualizar as variáveis do servidor.
   - Implemente a lógica para gerar os alarmes e eventos conforme as regras.
   - Habilite e configure o histórico para as variáveis e eventos designados.
   - Implemente a tarefa de registro no LDS.
4. **Serviço de API**:
   - Desenvolva uma API RESTful com FastAPI.
   - Crie endpoints para:
     - `GET /api/v1/motor/latest`: Retornar o último estado conhecido do motor.

        - `GET /api/v1/motor/history/{variable}` : Retornar dados históricos de uma variável com parâmetros de tempo.
        - `GET /api/v1/motor/alarms` : Retornar o histórico de alarmes.
    - Garanta que a documentação Swagger (`/docs`) seja gerada automaticamente.

5. **Testes e Documentação**:

    - Escreva testes unitários com `pytest` para as principais lógicas de negócio de cada serviço.
    - Os testes unitários devem superar 70% de cobertura.
    - Crie um script de teste E2E que simule o fluxo completo: publicação MQTT -> verificação na API -> validação no OPC UA.
    - Documente seu código utilizando docstrings para posterior geração com Pydoc.
    - A documentação da API deve ser atualizada e os modelos de requisição e reposta disponíveis no /docs

6. **Visualização de Dados**:

    - Configure uma instância do Grafana (pode ser via Docker Compose).
    - Crie um Data Source do tipo "JSON API" para conectar o Grafana à sua API FastAPI.
    - Monte um dashboard simples com pelo menos um gauge (para um valor em tempo real) e um gráfico de série temporal (para um valor histórico).

Dicas de Ferramentas

- **OPC UA SDK**: Python `asyncua` .
- **Cliente MQTT**: Python `gmqtt` .
- **API Framework**: `FastAPI` .
- **Message Broker**: `RabbitMQ` (com a lib `aio-pika`).
- **Testes**: `pytest` e `TestClient` do FastAPI.
- **Ferramenta de Teste OPC UA**: UaExpert.
- **Ferramenta de Teste MQTT**: MQTT Explorer.